Practical: 7

Practical 7: PHP Form Processing with Server-Side Validation and CSV/JSON File Storage
Problem Definition: Process submitted registration/contact form data using PHP. Validate and sanitize inputs on the server side and store records in CSV or JSON file format. Display success/error messages.

A. Implementation & Output Screenshots

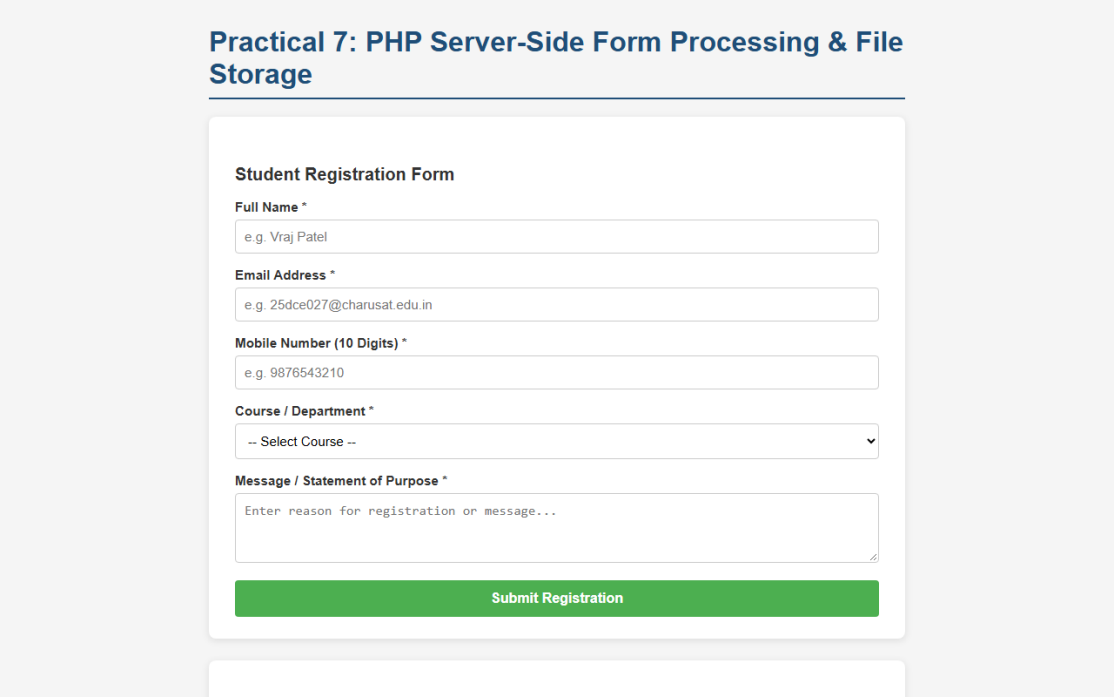


Figure 7.1: Registration / Contact Form Interface (Form Inputs and Validation Controls)

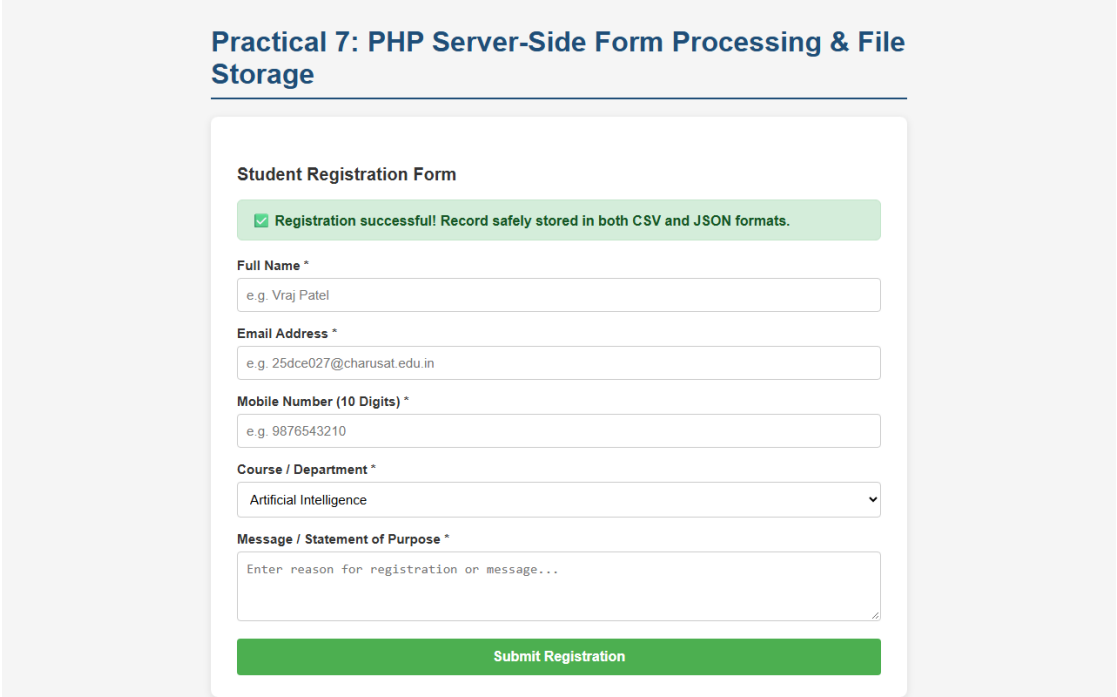


Figure 7.2: Successful Form Submission with Server-Side Validation and Success Message

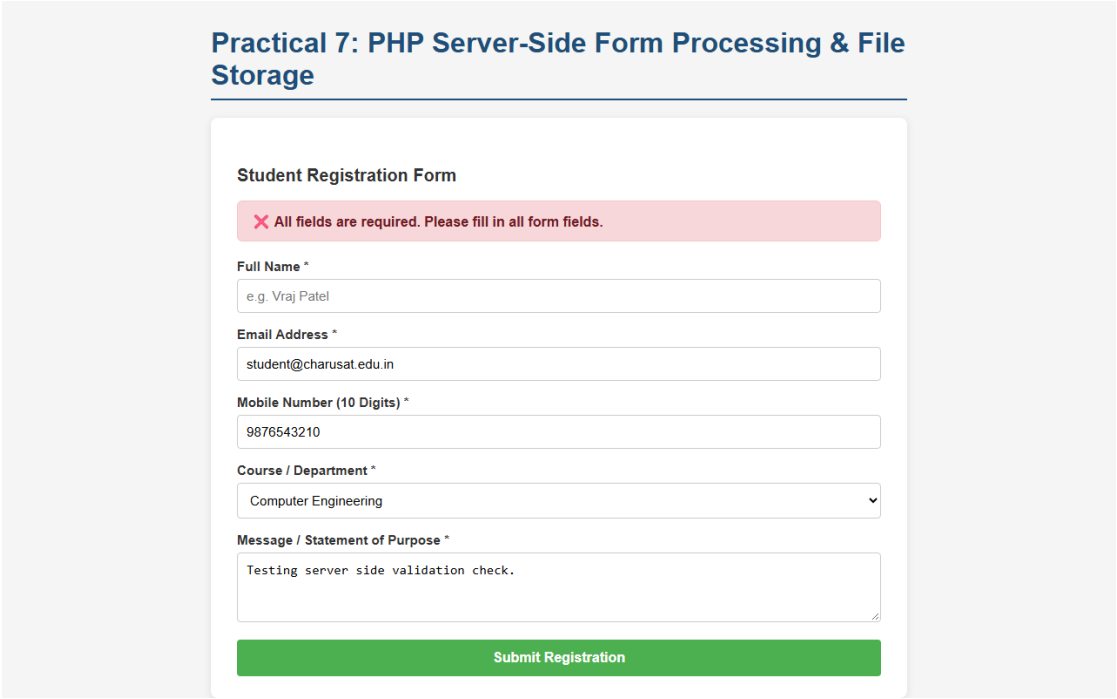


Figure 7.3: Server-Side Validation Error on Empty / Missing Required Fields

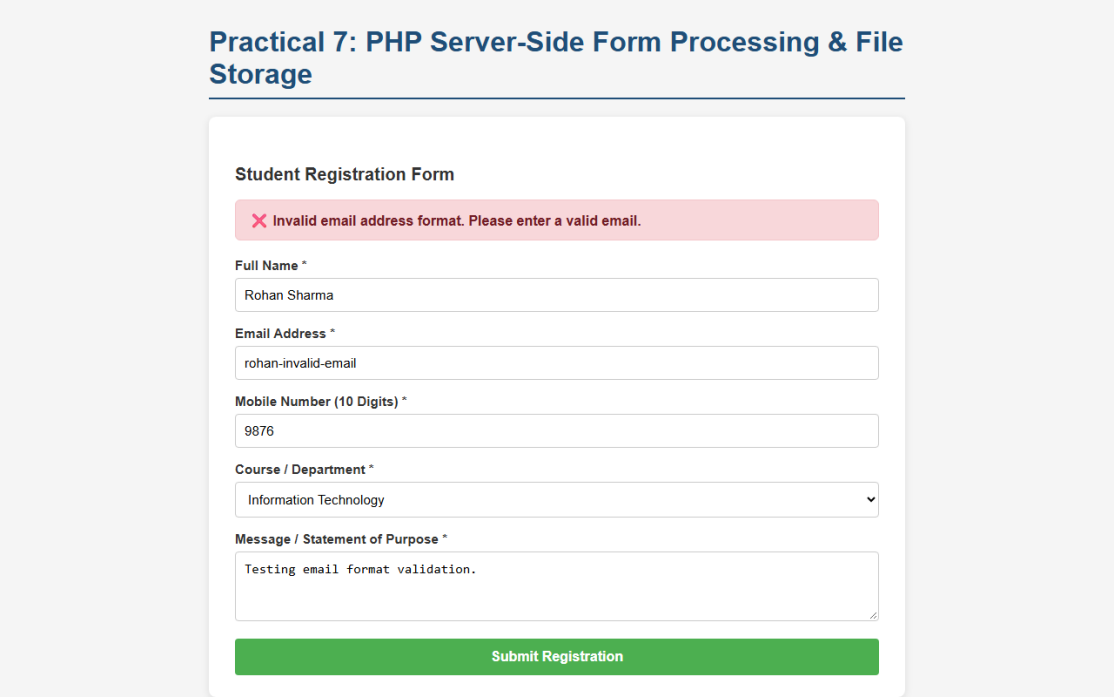


Figure 7.4: Server-Side Validation Error on Invalid Email and Mobile Format

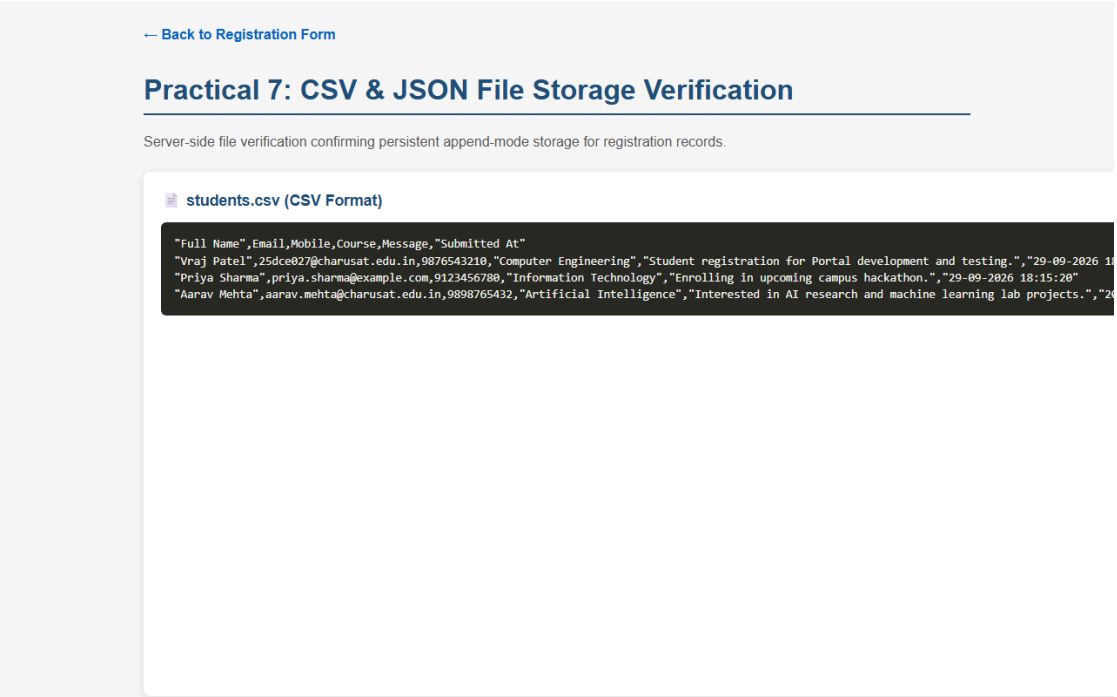


Figure 7.5: Persistent CSV and JSON File Storage Verification on Server

B. GitHub Link

https://github.com/VRAJ-710/studenthub-portal

C. Key Questions & Analysis (Theory)

1. Is the form submitted using POST?

Yes. The form is submitted using the HTTP POST method (method="POST" action=""). Using POST is crucial for registration and data collection forms because inputs are transmitted inside the HTTP request entity body rather than being appended as visible query parameters to the URL (as with GET). This prevents sensitive personal data (e.g., mobile numbers, emails, and personal messages) from being recorded in browser histories, web server access logs, or proxy caches. On the server side, PHP safely verifies the method using $_SERVER['REQUEST_METHOD'] === 'POST' and verifies the submission token.

2. Are inputs validated and sanitized server side?

Yes. Full validation and sanitization are strictly implemented on the server side prior to processing:
• Server-Side Validation: Required fields are tested using empty() to ensure no missing inputs. The email address is validated with PHP's built-in filter_var($email, FILTER_VALIDATE_EMAIL), which enforces strict RFC format compliance. The mobile number is verified via regular expression preg_match('/^[0-9]{10}$/', $mobile) to confirm exactly 10 numeric digits.
• Server-Side Sanitization: All inputs are trimmed of extraneous surrounding whitespace using trim(). For storage and output safety, strings are sanitized against Cross-Site Scripting (XSS) and HTML injection using htmlspecialchars($input, ENT_QUOTES, 'UTF-8').

3. Is file writing handled safely?

Yes. File operations are handled with multiple defensive layers to ensure data integrity:
• Exclusive File Locking (flock): To prevent concurrency race conditions when multiple users submit forms at the same time, PHP acquires an exclusive write lock (flock($fp, LOCK_EX)) before modifying files and releases it (flock($fp, LOCK_UN)) prior to closing with fclose().
• CSV Storage Safety: The fputcsv() function is used to automatically escape commas, double quotes, and line breaks according to RFC 4180 CSV specifications.
• JSON Storage Safety: JSON files are read under exclusive lock, decoded to PHP arrays, appended with the sanitized record, and rewritten cleanly using json_encode($data, JSON_PRETTY_PRINT).
• Safe Initialization: File existence is verified, and files are automatically initialized with appropriate headers/arrays if they do not already exist.

4. Are success and error responses displayed clearly?

Yes. The interface renders clear, accessible, and conditionally styled alert messages based on server-side processing results:
• Success Messages: Upon valid submission and successful file write, a green alert box (.alert-success) displays 'Registration successful! Record safely stored in both CSV and JSON formats.', and the new record immediately reflects in the stored records table.

• Error Messages: When any validation rule is violated (empty field, invalid email syntax, or non-10-digit mobile), a prominent red alert box (.alert-error) displays a specific, informative error message explaining exactly what failed. Valid input values are preserved in the form fields to ensure a smooth user experience.

D. Challenges Faced

- Preventing file corruption and race conditions during simultaneous form submissions by correctly applying flock() exclusive locking.
- Ensuring complete server-side input validation and sanitization that remains secure even if client-side browser validations are bypassed or disabled.
- Maintaining dual persistence pipelines to synchronize tabular CSV generation (fputcsv) alongside hierarchical JSON arrays (json_encode).

E. Learning Outcomes

- Implemented robust server-side form processing using the HTTP POST method.
- Mastered server-side validation using filter_var() and regular expressions, and sanitization using htmlspecialchars() to protect against injection attacks.
- Handled safe, atomic file writing with exclusive locking (flock) to persistently store records in CSV and JSON file formats.
- Constructed clear, dynamic user feedback mechanisms with conditional success and descriptive error messages.
- Gained practical experience implementing small data collection systems and backend form processors with file-based storage.

Faculty Remarks: